

A tested starting point

Stage 0 audit | 20 September 2026 | finite-dimensional open systems

Decision: hand this baseline to Fable for the next bounded implementation turn. The Mathlib-only representation probe builds, its ten theorem contracts elaborate, and all fifteen exported declarations satisfy the project axiom policy.

This is the foundation pass agreed after the broader 39-area research plan. It turns the selected open-systems pilot into a small, reproducible Lean project. It does not settle the wider materials, quantum algorithms, quantum information, or operations-research inventory.

Question	Observed result
Does the selected source compile?	Yes. The release module and umbrella build. The verifier also re-elaborates the release source explicitly.
Are the intended statements exposed?	Yes. Ten direct consumer checks compile in Audit/Contracts.lean.
What assumptions enter the proof terms?	All 15 exports report only propext, Classical.choice, and Quot.sound.
Can the gate detect bad cases?	15 control, rejection, and parser cases pass, including a forbidden transitive axiom and an added False premise.
Are dependency revisions known?	All 9 git dependencies match lake-manifest.json and have clean tracked files.
Has this run on your workstation?	No. The observed build is in an Ubuntu 24.04.3 x86_64 container. Local reproduction is Fable's first task.

What you receive

The source archive contains the pinned project, proof source, consumer contracts, verification scripts, raw evidence, architecture decisions, a completed audit, and NEXT_FABLE_TASK.md. The toolchain and dependency caches are intentionally fetched from their recorded upstream sources during reproduction.

No mathematical novelty or upstream acceptance is claimed. The value of this milestone is a checked representation, explicit theorem boundaries, and an audit process that Fable can extend one milestone at a time.

Representation and dependencies

One small mathematical base

The project uses Lean 4.34.0 and Mathlib v4.34.0 at the exact revision below. Mathlib is the only direct mathematical dependency; its own transitive packages remain pinned in the lockfile.

leanprover/lean4:v4.34.0

Mathlib: 5ed2965256430c3649e86755f9576b54eca72435

Choice	Meaning and reason
QubitMatrix	Matrix (Fin 2) (Fin 2) over the complex numbers. Basis labels are 0 and 1.
IsDensity rho	Exactly Mathlib positive semidefiniteness and $\text{trace}(\text{rho}) = 1$. This is a thin predicate over Mathlib matrices.
basisProjector i	A diagonal indicator, proved equal to $\text{Matrix.single } i \ i$. Both basis choices are densities and are distinct.
diagonalState q	$\text{diag}(1-q, q)$, with q real and interpreted as the population of basis state 1. Density validity requires $0 \leq q \leq 1$.
krausMap K X	The finite sum of $K_j X K_j^H$. Rectangular Kraus matrices are allowed. Normalization is a separate premise.

Why QICLean is a reference at this stage

Source inspection at revision af5430a1bb7050b86e7520035eccfa7e1e3249d6 found matching density semantics, a useful Kraus trace-preservation theorem, and a bridge from Kraus representation to all-finite-ancilla positivity. These are candidates for later reuse. QICLean was neither imported nor built in this pass.

That snapshot targets Lean/Mathlib 4.35.0-rc1 and has additional package requirements. The selected elementary probe already fits Mathlib 4.34.0. This is a scope and compatibility decision, not an adverse conclusion about QICLean. Its extra package requirements were not established as proof dependencies of the small trace theorem.

Convention to preserve

In the planned two-jump model, a is the rate 0 to 1 and b is the rate 1 to 0. The stationary candidate will be $\text{diagonalState}(a/(a+b))$, when the rate hypotheses hold. A later four-Kraus formula uses $\text{populationZero}=b/(a+b)$. Keep populationOne and populationZero distinct. No rate-dependent theorem has yet been proved.

The exact mathematical surface

All declarations below are in FormalScience.Stage0. Five additional exported definitions or abbreviations fix QubitMatrix, IsDensity, basisProjector, diagonalState, and krausMap. The ten theorem contracts are:

Declaration	Required mathematical content
basisProjector_eq_single	For each i in $\text{Fin } 2$, the chosen basis projector equals the standard matrix unit E_{ii} .
basisProjector_isDensity	Every such basis projector is positive semidefinite and has trace one.
diagonalState_isDensity	For every real q with $0 \leq q$ and $q \leq 1$, $\text{diag}(1-q, q)$ is positive semidefinite and has trace one.
diagonalState_zero	The $q=0$ endpoint is the basis projector at 0.
diagonalState_one	The $q=1$ endpoint is the basis projector at 1.
basisProjector_zero_ne_one	The two concrete density witnesses differ.
krausMap_trace	For arbitrary finite input, output, and Kraus index types, $\text{trace}(\sum K_j X K_j^H) = \text{trace}((\sum K_j^H K_j) X)$.
krausMap_trace_preserving	If $\sum K_j^H K_j = I$, the finite sum preserves trace for every input matrix X .
identityKraus_complete	A singleton identity Kraus family satisfies the completeness premise.
identityKraus_apply	That singleton family acts as the identity on every qubit matrix.

Quantifiers and assumptions matter

The Kraus theorems quantify over arbitrary X , with no density, positivity, Hermiticity, or normalization requirement on X . The trace identity needs only finite index types. The trace-preservation statement additionally uses decidable equality on the input basis to express the identity matrix. Its completeness premise is explicit.

The basis and identity examples establish nonvacuity. They do not replace the general statements. The basis projector equality is checked, but no separate rank-one or purity theorem is included.

Audit/Contracts.lean states the density conclusions using Mathlib PSD and trace directly, and the Kraus conclusions using explicit finite sums. Each check directly applies the required public theorem. The printed export types are preserved with the build evidence.

Verification that checks the claim

Four complementary checks

Layer	What it establishes in this snapshot
Dependency and coverage checks	The compiler version, Mathlib revision, and all locked git revisions match. The release module set agrees with the manifest and is reachable from the umbrella.
Build and source elaboration	Lake builds the release targets. A direct Lean pass re-elaborates the release source even when incremental artifacts are current.
Independent consumer signatures	All ten promised theorem types remain usable with their stated quantifiers and premises. Underlying density and trace formulas are spelled out.
Transitive axiom reports	Every one of the 15 listed exports, including definitions, has an exact parsed axiom report within the accepted set. Missing or malformed reports fail the gate.

The gate was tested against bad cases

All 15 cases pass their expected outcome: an axiom-free control; an allowed-propext control; direct and transitive custom axioms; sorryAx; a missing declaration; an axiom-clean False-premise control; a valid signature control; rejection of the changed False-premise signature; and six missing, duplicate, extra, malformed, or misleading-name report cases.

An axiom-clean theorem can still be the wrong theorem. The False-premise fixture has no forbidden axioms, but cannot satisfy the original direct consumer signature. This is why the handoff retains both signature checks and axiom checks.

Negative fixture files intentionally contain rejected declarations and are outside the release module set. They are evidence that the gate fails for the intended reasons. Do not count them as project proofs or repair them into passing declarations.

What still needs review

The export list is authoritative and human-reviewed; the script does not automatically enumerate every possible public declaration. Module coverage is checked, but newly added exports still require review and manifest updates. The current list covers the five definitions and ten theorems in the release source.

This is an ordinary proof and reproducibility gate, not malicious-code hardening. Reviewers must inspect definitions, intended mathematical meaning, and changes to the verification scripts. The project also trusts its official Lean toolchain and cached third-party artifacts.

Fable's next implementation turn

Active milestone: finite dissipator algebra. Reproduce Stage 0 first, then add one small module under FormalScience/OpenSystems/. Return the result for the next audit before extending to stationary states or dynamics.

The mathematical contract

Use the existing finite complex matrix representation. With V^H denoting conjugate transpose, define the dissipator by:

$$D[V](X) = V X V^H - (1/2) * (V^H V X + X V^H V)$$

In Lean, express the one-half factor as complex scalar multiplication. Choose direct algebraic lemmas or a bundled complex-linear map according to the simplest useful consumer API.

- Prove additivity and complex homogeneity in X , or provide a complex-linear map with the displayed defining formula.
- Prove $\text{trace}(D[V](X)) = 0$ for every V and every X .
- Prove Hermiticity preservation: if X is Hermitian, then $D[V](X)$ is Hermitian, for every V .
- Specialize these results to $V=E_{10}$ and $V=E_{01}$ using Mathlib matrix-unit multiplication and adjoint facts, or thin checked adapters.

Do not assume V is Hermitian or unitary: the intended jump matrices satisfy neither restriction. A dissipator is a generator component; this assignment does not claim it is a positive or completely positive channel.

Required handoff

Preserve existing theorem signatures and conventions. Update the umbrella imports, release module/export manifest, and direct consumer contracts. Run both validation scripts and return the source changes, exact new statements, fresh logs, dependencies, decisions, and any unresolved issue using `AUDIT_HANDOFF_TEMPLATE.md`.

How the implementation-audit loop will work

Fable owns the bounded implementation. The next audit checks correspondence with the informal contract, quantifiers, definitions, dependency scope, build coverage, and transitive axioms. Compilation alone does not select the next mathematical claim. If Fable encounters a missing API, it should report the smallest blocking goal before expanding the project.

The broad roadmap remains available for future choices. The stationary system, continuous-time evolution, complete positivity, convergence, generic GKSL, entropy, circuits, and OR branches are outside this next turn.

Reproduce and inspect the evidence

Unpack Formal_Science_Stage0_Handoff.zip and open its formal-science directory in Cursor. With Git, Python 3, and a working Lean/elan installation, run these commands from the project root. The portable instructions are also in README.md.

```
elan toolchain install leanprover/lean4:v4.34.0
lake exe cache get Mathlib.Analysis.Complex.Basic \
  Mathlib.LinearAlgebra.Matrix.PosDef \
  Mathlib.LinearAlgebra.Matrix.Trace \
  Mathlib.Tactic.FinCases Mathlib.Tactic.NormNum
python3 scripts/verify.py
python3 scripts/test_verify.py
```

The first Lake command materializes the locked dependencies and fetches the targeted cache. Preserve lake-manifest.json; do not run a broad lake update as a proof repair. The scripts also accept --lake /absolute/path/to/lake for an isolated toolchain.

Evidence path in the project	Use
evidence/stage0/verification/	Passed summary, exact commands and exit codes, source hashes, signatures, axiom queries, and raw output.
evidence/stage0/gate-tests/	15-case summary, raw output, and the deliberately invalid Lean fixtures.
evidence/stage0/setup/	Download, dependency/cache setup, and initial build logs, including resolved failures.
evidence/stage0/research/	Source reconnaissance and semantic review. QICLean remains source-inspected only.
evidence/stage0/environment.json	Observed OS, toolchain, compiler commit, archive checksum, and cache boundary.
SOURCE_MANIFEST.json	SHA-256 inventory of all packaged project files except the manifest itself.

Observed environment and trust boundary

The local run used Ubuntu 24.04.3 x86_64, Python 3.12.14, Lean 4.34.0, and Lake 5.0.0-src+293d5d0. The toolchain came from the official Lean release. Its archive checksum is recorded in environment.json. All nine dependency checkouts matched their locked revisions and had clean tracked files.

Official precompiled Mathlib artifacts were used for the selected import closure. This was not a full source rebuild of Mathlib. The 2188-job Lake build summary includes cached dependency work and must not be described as 2188 newly proved modules. The local release source and consumer checks were elaborated with the pinned compiler.

Release boundary and provenance

What the successful result means

Stage 0 closes the representation and trace-algebra reconnaissance task. The selected density examples and Kraus trace results have proof terms accepted by the pinned Lean environment under the recorded foundational-axiom policy. The snapshot is ready for a local reproduction and the finite dissipator milestone.

It does not prove a stationary state, uniqueness, a semigroup, complete positivity, trace-norm convergence, or a generic Lindblad theorem. No quantum algorithm or OR theorem is claimed here. Each future claim needs its own contract, prerequisites, hypotheses, and audit.

Resolved implementation issues

Initial setup required preserving the official toolchain bin directory on PATH for the cache helper. The final probe imports Mathlib.Analysis.Complex.Basic explicitly and uses Complex.zero_le_real.mpr for nonnegative real scalars. A basis-equality simplification loop was replaced by explicit finite cases. These were resolved locally without changing third-party sources or the intended statements.

The handoff review also corrected Lake executable resolution to preserve elan proxy symlink names. The packaged verifier was rerun after that repair. The original source-reconnaissance note includes a correction for its provisional Complex.ofReal_nonneg suggestion.

Source provenance

The project probe uses Mathlib APIs with locally written definitions and proofs; no QICLean proof body was copied. The QICLean audit identifies candidate declarations and import boundaries only. Dependency sources and their notices remain upstream and are fetched at locked revisions. The artifact is a source archive with a hash manifest, not a published repository or an accepted Mathlib contribution.

[Lean 4.34.0 official release](#)

[Pinned Mathlib source: matrix PSD](#)

[Pinned Mathlib source: matrix trace](#)

[Pinned QICLean snapshot, inspected only](#)

[Lean reference: validating proofs](#)

[Mathlib project setup guide](#)

Primary evidence for the build claims is the preserved local output, not the linked documentation. The earlier comprehensive research plan remains the portfolio roadmap. This audit supersedes only provisional Stage 0 implementation choices and records the next bounded assignment.